

05506026 Digital Image Processing (Java)

Lab 9

Objectives

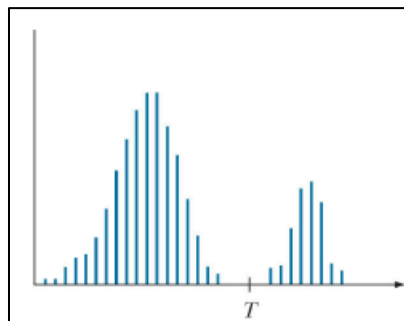
1. Understand how to perform global thresholding method using Otsu's
2. Understand how to perform canny edge detector

Thresholding

Thresholding is a simple global operation in which we separate the image into 2 or more segments. Mostly, thresholding will be divided into 2 segments. Therefore, the thresholding method is also known as binarization method because we turn a grayscale image to a binary image.

The basic thresholding method can be simply implemented by only checking if the intensity of each pixel is above the defined threshold value or not without considering its context. If above, turn that pixel into a white pixel. Else, turn it into a black pixel.

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{if } f(x, y) \leq T \end{cases}$$



```

ImageManager.java
class ImageManager
{
    ...
    public void thresholding(int threshold)
    {
        if (img == null)    return;

        convertToGrayscale();

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int color = img.getRGB(x, y);
                int gray = color & 0xff;

                gray = gray < threshold? 0: 255;

                color = (gray << 16) | (gray << 8) | gray;

                img.setRGB(x, y, color);
            }
        }
        ...
    }
}

```

Below is Mandril.bmp with the threshold value set at 50, 100, and 150 respectively.



However, the challenging part is to define an appropriate threshold value. There are a few methods to calculate this value. One of those called Otsu's method

The threshold value is a number which is between 0 to L-1 in image with L different intensity values. However, the value is picked by user which can give errors in thresholding result. Otsu's method measures between-class variance to pick the threshold value where they can best separate each class.

We can implement Otsu's method as follow and thresholding method above.

```

ImageManager.java
class ImageManager
{
    ...
    public void otsuThreshold()
    {
        if (img == null)    return;

        convertToGrayscale();

        int[] histogram = new int[256];

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int color = img.getRGB(x, y);
                int gray = color & 0xff;

                histogram[gray]++;
            }
        }

        float[] histogramNorm = new float[histogram.length];
        float pixelNum = width * height;

        for (int i = 0; i < histogramNorm.length; i++)
        {
            histogramNorm[i] = histogram[i] / pixelNum;
        }

        float[] histogramCS = new float [256];
        float[] histogramMean = new float[256];
    }
}

```

```

for (int i = 0; i < 256; i++)
{
    if (i == 0)
    {
        histogramCS[i] = histogramNorm[i];
        histogramMean[i] = 0;
    }
    else
    {
        histogramCS[i] = histogramCS[i - 1] + histogramNorm[i];
        histogramMean[i] = histogramMean[i - 1] + histogramNorm[i] * i;
    }
}

float globalMean = histogramMean[255];
float max = Float.MIN_VALUE;
float maxVariance = Float.MIN_VALUE;
int countMax = 0;

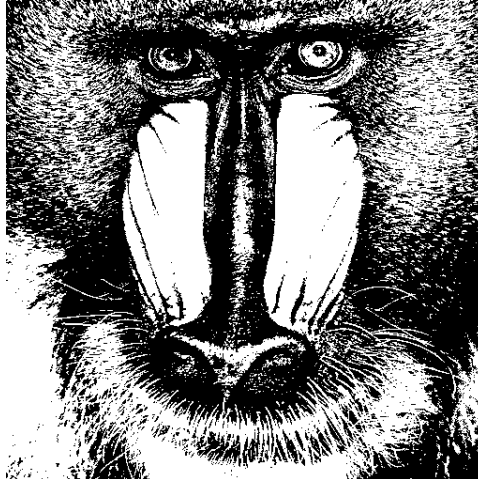
for (int i = 0; i < 256; i++)
{
    float variance = (float)Math.pow(globalMean * histogramCS[i] - histogramMean[i],
2) / (histogramCS[i] * (1 - histogramCS[i]));

    if (variance > maxVariance)
    {
        maxVariance = variance;
        max = i;
        countMax = 1;
    }
    else if (variance == maxVariance)
    {
        countMax++;
        max = ((max * (countMax - 1)) + i) / countMax;
    }
}

thresholding((int)Math.round(max));
}
...
}

```

The result of using Otsu's method on Mandril.bmp is as follow.



Linear Spatial Filter

In past labs, we have learnt on some spatial filter before such as averaging filter. There are many more linear filters which all are identical in calculation. As we will need to perform several different linear filters from this point onwards, it is a good idea to prepare a generalised version of linear spatial filtering method.

We can implement linear spatial filtering using code as follow which will receive an array of double values and an integer to indicate the size.

```

ImageManager.java
class ImageManager
{
    ...
    public void linearSpatialFilter(double[] kernel, int size)
    {
        if (img == null)    return;

        if (size % 2 == 0)
        {
            System.out.println("Size Invalid: must be odd number!");
            return;
        }

        BufferedImage tempBuf = new BufferedImage(width, height, img.getType());

        for (int y = 0; y < height; y++)
        {

```

```

        for (int x = 0; x < width; x++)
        {
            double sumRed = 0, sumGreen = 0, sumBlue = 0;

            for (int i = y - size/2; i <= y + size/2; i++)
            {
                for (int j = x - size/2; j <= x + size/2; j++)
                {
                    if (i >= 0 && i < height && j >= 0 && j < width)
                    {
                        int color = img.getRGB(j, i);
                        int r = (color >> 16) & 0xff;
                        int g = (color >> 8) & 0xff;
                        int b = color & 0xff;

                        sumRed += r * kernel[(i - (y - size/2)) * size + (j - (x - size/2))];
                        sumGreen += g * kernel[(i - (y - size/2)) * size + (j - (x - size/2))];
                        sumBlue += b * kernel[(i - (y - size/2)) * size + (j - (x - size/2))];
                    }
                }
            }

            sumRed = sumRed > 255 ? 255 : sumRed;
            sumRed = sumRed < 0 ? 0 : sumRed;

            sumGreen = sumGreen > 255 ? 255 : sumGreen;
            sumGreen = sumGreen < 0 ? 0 : sumGreen;

            sumBlue = sumBlue > 255 ? 255 : sumBlue;
            sumBlue = sumBlue < 0 ? 0 : sumBlue;

            int newColor = ((int)sumRed << 16) | ((int)sumGreen << 8) | (int)sumBlue;

            tempBuf.setRGB(x, y, newColor);
        }
    }

    for (int y = 0; y < height; y++)
    {
        for (int x = 0; x < width; x++)
        {
            img.setRGB(x, y, tempBuf.getRGB(x, y));
        }
    }
    ...
}

```

Canny Edge Detector

Canny Edge Detector is one of edge detection algorithms which is more complex but better performance in general. There are three basic objectives of this algorithm: -

1. Low error rate
2. Edge points are located as close as possible to the centre of true edges
3. Return only single edge point

Canny Edge Detector has 4 steps which are

1. Smooth the image using Gaussian filter
2. Find intensity gradient magnitude and direction
3. Apply non-maxima suppression
4. Apply double threshold (hysteresis thresholding)

We will separate each step in this lab explanation with some result images which WILL NOT actually be produced during the method. The images are saved for you to be better in understanding the current process of the algorithm.

First, we create a method to perform Canny Edge Detector with two threshold parameters for step 4.

```
ImageManager.java
class ImageManager
{
    ...
    public void cannyEdgeDetector(int lower, int upper)
    {
    }
    ...
}
```

These parameters will not be used until the very end of the algorithm.

Next, we apply a Gaussian filter on our image to smooth it and be prepared for the next process. A 5 x 5 filter is widely used in digital image processing for this step. We will use linear spatial filtering method we created in the previous section and convert the image to grayscale afterwards.

```

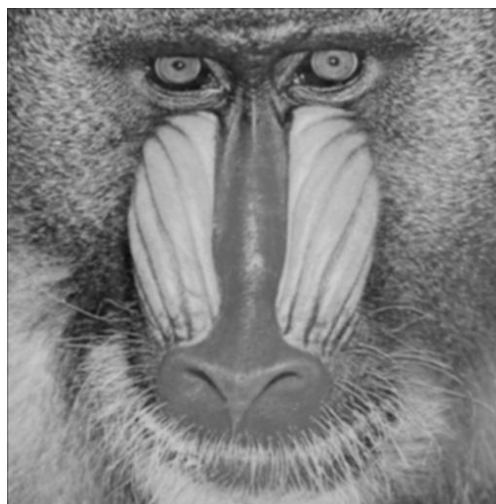
ImageManager.java
class ImageManager
{
    ...
    public void cannyEdgeDetector(int lower, int upper)
    {
        //Step 1 - Apply 5 x 5 Gaussian filter

        double[] gaussian =
{2.0 / 159.0, 4.0 / 159.0, 5.0 / 159.0, 4.0 / 159.0, 2.0 / 159.0,
 4.0 / 159.0, 9.0 / 159.0, 12.0 / 159.0, 9.0 / 159.0, 4.0 / 159.0,
 5.0 / 159.0, 12.0 / 159.0, 15.0 / 159.0, 12.0 / 159.0, 5.0 / 159.0,
 4.0 / 159.0, 9.0 / 159.0, 12.0 / 159.0, 9.0 / 159.0, 4.0 / 159.0,
 2.0 / 159.0, 4.0 / 159.0, 5.0 / 159.0, 4.0 / 159.0, 2.0 / 159.0};
        linearSpatialFilter(gaussian, 5);

        convertToGrayscale();
    }
    ...
}

```

This will produce an image as follow.



Step 2, we will find the intensity gradient magnitude and direction in the entire image. We will use Sobel filter to detect the edge in both X and Y direction, which we can use these two values of each pixel to calculate gradient magnitude and direction as follow.

$$M_s(x, y) = \|\nabla f_s(x, y)\| = \sqrt{g_x^2(x, y) + g_y^2(x, y)}$$

$$\alpha(x, y) = \tan^{-1} \left[\frac{g_y(x, y)}{g_x(x, y)} \right]$$

The code is implemented as follow.

```

ImageManager.java
class ImageManager
{
    ...
    public void cannyEdgeDetector(int lower, int upper)
    {
        //Step 1 - Apply 5 x 5 Gaussian filter
        ...
        //Step 2 - Find intensity gradient
        double[] sobelX = { 1, 0, -1, 2, 0, -2, 1, 0, -1};
        double[] sobelY = { 1, 2, 1, 0, 0, 0, -1, -2, -1};

        double[][] magnitude = new double[height][width];
        double[][] direction = new double[height][width];

        for (int y = 3; y < height - 3; y++)
        {
            for (int x = 3; x < width - 3; x++)
            {
                double gx = 0, gy = 0;
                for (int i = y - 1; i <= y + 1; i++)
                {
                    for (int j = x - 1; j <= x + 1; j++)
                    {

```

```

        if (i >= 0 && i < height && j >= 0 && j < width)
        {
            int color = img.getRGB(j, i);
            int gray = color & 0xff;

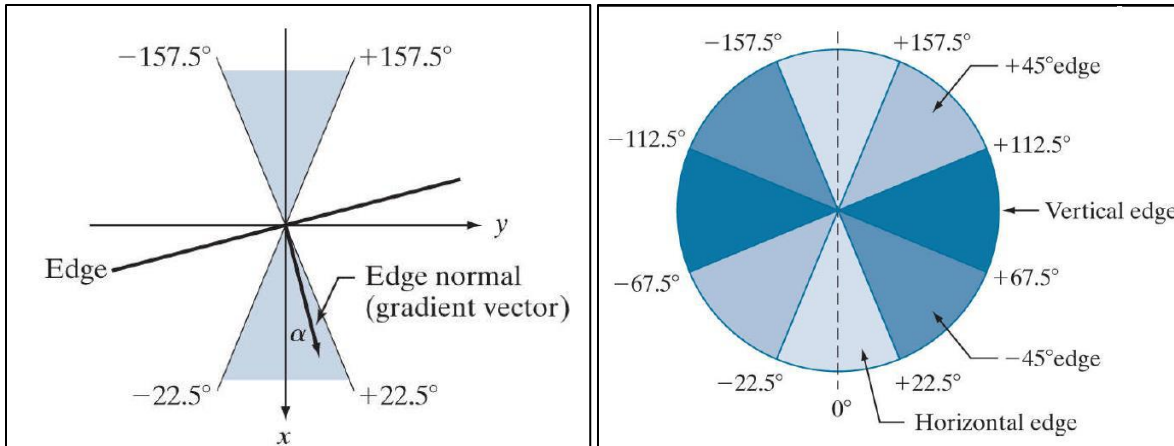
            gx += gray * sobelX[(i - (y - 1)) * 3 + (j - (x - 1))];
            gy += gray * sobelY[(i - (y - 1)) * 3 + (j - (x - 1))];
        }
    }
    magnitude[y][x] = Math.sqrt(gx * gx + gy * gy);
    direction[y][x] = Math.atan2(gy, gx) * 180 / Math.PI;
}
...
}

```

This will produce an image as follow for magnitude.



Step 3, apply non-maxima suppression. Meaning that every pixel will be checked along its direction calculated in step 2 that it is the local maximum value (compare to the neighbour pixels in the direction). If it is not, the value will be suppressed (convert to zero). However, the direction can be infinite which can cause expensive processing. Therefore, it is rounded to four discrete directions instead.



The code is implemented as follow.

```

ImageManager.java
class ImageManager
{
    ...
    public void cannyEdgeDetector(int lower, int upper)
    {
        //Step 1 - Apply 5 x 5 Gaussian filter
        //Step 2 - Find intensity gradient
        ...
        //Step 3 - Nonmaxima Suppression
        double[][] gn = new double[height][width];
        for (int y = 3; y < height - 3; y++)
        {
            for (int x = 3; x < width - 3; x++)
            {
                int targetX = 0, targetY = 0;
                //find closest direction
                if (direction[y][x] <= -157.5)
                { targetX = 1; targetY = 0; }
                else if (direction[y][x] <= -112.5)
                { targetX = 1; targetY = -1; }
                else if (direction[y][x] <= -67.5)
                { targetX = 0; targetY = 1; }
                else if (direction[y][x] <= -22.5)
                { targetX = 1; targetY = 1; }
                else if (direction[y][x] <= 22.5)
                { targetX = 1; targetY = 0; }
            }
        }
    }
}

```

```

else if (direction[y][x] <= 67.5)
{   targetX = 1;   targetY = -1;   }
else if (direction[y][x] <= 112.5)
{   targetX = 0;   targetY = 1;   }
else if (direction[y][x] <= 157.5)
{   targetX = 1;   targetY = 1;   }
else
{   targetX = 1;   targetY = 0;   }

if (y + targetY >= 0 && y + targetY < height &&
    x + targetX >= 0 && x + targetX < width &&
    magnitude[y][x] < magnitude[y + targetY][x + targetX])
{
    gn[y][x] = 0;
}
else if (y - targetY >= 0 && y - targetY < height &&
    x - targetX >= 0 && x - targetX < width &&
    magnitude[y][x] < magnitude[y - targetY][x - targetX])
{
    gn[y][x] = 0;
}
else
{
    gn[y][x] = magnitude[y][x];
}
}
...
}

```

This will produce an image as follow



The last step, we will apply hysteresis thresholding which is one of double thresholding methods. The algorithm will use values from two parameters and a recursive function as follow.

```

ImageManager.java
class ImageManager
{
    ...
    public void cannyEdgeDetector(int lower, int upper)
    {
        //Step 1 - Apply 5 x 5 Gaussian filter
        //Step 2 - Find intensity gradient
        //Step 3 - Nonmaxima Suppression
        ...
        //Step 4 - Hysteresis Thresholding

        //set back first
        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int newGray = (int) gn[y][x];
                newGray = newGray > 255? 255: newGray;
                newGray = newGray < 0? 0: newGray;

                int newColor = (newGray << 16) | (newGray << 8) | newGray;
                img.setRGB(x, y, newColor);
            }
        }

        // upper threshold checking with recursive
        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int checking = img.getRGB(x, y) & 0xff;
                if (checking >= upper)
                {
                    checking = 255;
                    int newColor = (checking << 16) | (checking << 8) | checking;
                    img.setRGB(x, y, newColor);
                }
            }
        }
    }
}

```

```

        hystConnect(x, y, lower);
    }
}

//clear unwanted values
for (int y = 0; y < height; y++)
{
    for (int x = 0; x < width; x++)
    {
        int checking = img.getRGB(x, y) & 0xff;

        if (checking != 255)
        {
            int newColor = (0 << 16) | (0 << 8) | 0;
            img.setRGB(x, y, newColor);
        }
    }
}

private void hystConnect(int x, int y, int threshold)
{
    int value = 0;
    for (int i = y - 1; i <= y + 1; i++)
    {
        for (int j = x - 1; j <= x + 1; j++)
        {
            if ((j < width) && (i < height) &&
                (j >= 0) && (i >= 0) &&
                (j != x) && (i != y))
            {
                value = img.getRGB(j, i) & 0xff;
                if (value != 255)
                {
                    if (value >= threshold)
                    {
                        int newColor = (255 << 16) | (255 << 8) | 255;
                        img.setRGB(j, i, newColor);
                        hystConnect(j, i, threshold);
                    }
                    else
                    {
                        int newColor = (0 << 16) | (0 << 8) | 0;
                        img.setRGB(j, i, newColor);
                    }
                }
            }
        }
    }
}

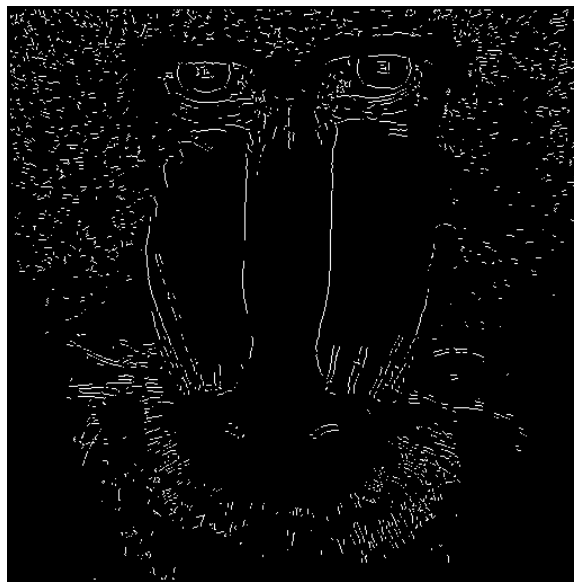
```

```

    }
    }
    }
    }
    }
    ...
    }

```

This will produce an image as follow, which is the result of Canny Edge Detector algorithm using 48 as the lower threshold and 128 as the upper threshold.



Quest 013

Implement the method of Canny Edge Detector and perform upper value as 180 and lower value as 100

The player who finishes this task will be awarded **25 EXP** and **1 Move**.

Deadline: 7 September 2025